INSIDE STLC ACADEMY

AI FOR QA TESTERS

AI Prompt Library (100 Prompts)

100 practical QA prompts across 16 categories — your everyday AI-assisted testing toolkit.

Introduction

This library gives you 100 ready-to-use QA prompts across 16 categories. Each is a starting point: add your real context, run it, and always review and verify the output. Treat AI as a fast drafting assistant — you remain the reviewer and decision-maker.

How to Use It

- Replace the [paste]/[context] placeholders with your real (non-sensitive) information.
- Refine with follow-ups ("add negative cases", "make it a table", "what did you miss?").
- Always verify the output against the real source before relying on it.
- Never paste confidential, customer or production data into public AI tools.

Requirements Analysis

1. Act as a senior QA analyst. Review this requirement for clarity, gaps and testability, and list the questions I should raise: [paste].
2. Identify any ambiguous or vague terms in this requirement and suggest measurable alternatives: [paste].
3. List assumptions a developer might make from this requirement that should be confirmed: [paste].
4. Highlight missing error-handling and edge conditions in this requirement: [paste].
5. Summarise this requirement in one sentence and list what is explicitly out of scope: [paste].
6. What non-functional requirements (performance, security, accessibility) are implied but not stated here? [paste].

Acceptance Criteria

1. Review these acceptance criteria for completeness and list missing scenarios: [paste].
2. Rewrite these acceptance criteria as clear Given/When/Then statements: [paste].
3. What negative and boundary cases are not covered by these acceptance criteria? [paste].
4. Are these acceptance criteria measurable and testable? Flag any that are not and suggest fixes: [paste].
5. Generate acceptance criteria for this user story, including at least one negative case: [paste].
6. Identify hidden assumptions in these acceptance criteria: [paste].

Test Cases

1. Generate test cases for this feature as a table (Title, Steps, Expected, Priority), covering valid, invalid and boundary inputs: [paste].
2. Using boundary value analysis, list the exact boundary inputs to test for this field: [paste].
3. Apply equivalence partitioning to this input and give one representative value per partition: [paste].
4. Review these test cases for duplication, gaps and unclear expected results: [paste].
5. Prioritise these test cases by risk and explain your reasoning: [paste].
6. Generate negative and edge-case test cases I likely missed for this feature: [paste].

Exploratory Testing

1. Draft three exploratory testing charters for this feature (explore X to discover Y using Z): [paste].
2. Generate user personas (new, power, careless, malicious, accessibility) to test this feature from different angles: [paste].
3. Brainstorm 20 "what could go wrong" scenarios for this feature, grouped by theme: [paste].
4. Suggest the highest-risk areas of this feature to explore first and why: [paste].
5. Turn these exploratory findings into clear, reproducible notes: [paste].
6. Suggest creative edge cases a scripted test plan would likely miss for this feature: [paste].

Regression Testing

1. Given this change, list existing areas that could be impacted and should be regression tested: [paste].
2. Draft a risk-based regression checklist for this release of changes: [paste].
3. Which critical user journeys should always be in our core regression suite? [context].
4. Suggest which of these tests are good automation candidates for regression and why: [paste].
5. Help me scope a regression pass when I only have limited time for this change: [paste].
6. Identify previously fixed defects that should be re-verified after this change: [paste].

API Testing

1. Explain what this API endpoint does, its inputs, outputs and status codes: [paste docs].
2. List API test scenarios for this endpoint: success, validation errors, auth failures, boundaries: [paste].
3. For each scenario, what status code and response should I expect? [paste].
4. Draft a valid and an invalid JSON request body for this endpoint: [paste].
5. What security and authorisation risks should I test for this endpoint? [paste].
6. Generate synthetic test data for this endpoint (use example.com, no real data): [paste].

Automation Support

1. Scaffold a Playwright test (Arrange-Act-Assert) for this scenario: [paste].
2. Suggest stable locators (test ids/roles) instead of these fragile selectors: [paste].
3. Explain this test failure / stack trace and suggest likely causes: [paste].
4. Refactor this test for readability and maintainability without changing what it verifies: [paste].
5. Why might this test be flaky, and how do I fix it properly? [paste].
6. Generate synthetic, data-driven test data for this automated test: [paste].

Bug Reports

1. Turn these rough notes into a structured bug report (title, env, steps, expected, actual): [paste].
2. Improve the clarity of this bug report without changing any facts: [paste].
3. Suggest a concise, descriptive title for this defect: [paste].

4. What information is missing from this bug report for a developer to reproduce it? [paste].
5. Suggest a severity and priority for this defect with reasoning (I will confirm): [paste].
6. Reduce these reproduction steps to the minimum that still triggers the bug: [paste].

Root Cause Analysis

1. Suggest possible root causes for this defect as hypotheses to investigate: [paste].
2. What variables should I change to isolate this bug (browser, data, account, network)? [paste].
3. Group these related defects and suggest a possible systemic cause: [paste].
4. Summarise this long defect thread into a clear problem statement: [paste].
5. What questions should I ask to get to the root cause of this issue? [paste].
6. Given this error pattern, what areas of the system are most likely involved? [paste].

Risk Assessment

1. Brainstorm risks for this feature across functional, security, performance and data: [paste].
2. Help me rate these risks by likelihood and impact (I will set the final ratings): [paste].
3. Which of these risks warrant the deepest testing, and why? [paste].
4. What could go wrong with this feature that the team may not have considered? [paste].
5. Translate these risks into a focused test plan outline: [paste].
6. Identify data-integrity risks in this feature: [paste].

Test Planning

1. Draft a concise test plan outline (scope, approach, entry/exit, risks) for this release: [paste].
2. Suggest entry and exit criteria for testing this feature: [paste].
3. Outline the test levels and types appropriate for this change: [paste].
4. Help me define what is in and explicitly out of scope for this release: [paste].
5. Draft a stakeholder-friendly test summary from these results: [paste].
6. Suggest meaningful QA metrics for this project and the decision each supports: [context].

User Stories

1. Rewrite this requirement as a clear user story (As a… I want… so that…): [paste].
2. Critique this user story against INVEST and suggest improvements: [paste].
3. Split this large user story into smaller, testable stories: [paste].
4. Generate acceptance criteria and test scenarios for this user story: [paste].
5. What clarifying questions should I raise about this user story in refinement? [paste].
6. Identify the testability concerns in this user story: [paste].

Documentation

1. Summarise these test results for stakeholders, leading with risk and impact: [paste].
2. Turn these notes into a clear test summary report: [paste].
3. Draft a concise README for this test project / portfolio piece: [paste].
4. Rewrite this technical update for a non-technical audience: [paste].
5. Create a short how-to for running these tests: [paste].
6. Summarise this module of work into three bullet points for a status update: [paste].

Leadership

1. Suggest process improvements from these defect trends to discuss in a retro: [paste].

2. Help me prepare talking points to advocate for more testing time using risk: [context].
3. Draft a coaching plan to help a junior tester improve bug-report quality: [context].
4. Suggest how to introduce risk-based testing to a team new to it: [context].
5. Identify testing bottlenecks these patterns suggest: [paste].
6. Draft a short quality update for leadership focused on outcomes: [paste].

Interview Preparation

1. Ask me five common QA interview questions and critique my answers one at a time.
2. Give a model answer to "what is the difference between severity and priority?" with an example.
3. Pose a scenario interview question and evaluate my structured response.
4. Help me turn this experience into a STAR-format interview story: [paste].
5. Generate likely interview questions for this QA job description: [paste].
6. Review my answer to "how would you test X?" for structure and coverage: [paste].

Prompt Engineering

1. Improve this prompt for clarity, context and a specific output format: [paste].
2. Turn this task into a reusable prompt template with placeholders: [paste].
3. Suggest the best role and output format for this QA prompt: [paste].
4. Critique this prompt and explain why its output was weak: [paste].
5. Add constraints to this prompt so the output is focused and concise: [paste].
6. Convert this one-line request into a structured role/context/task/format prompt: [paste].

Best Practices

- Give the AI a role, context, task and output format.
- Iterate — the first answer is a draft.
- Pair prompts with your own QA techniques to judge the output.

Common Mistakes

- Using vague one-line prompts with no context.
- Accepting output without verification (hallucinations).
- Pasting sensitive data into public tools.

> **INSIDE STLC PRO TIP**
>
> That is 96+ prompts to start from. Save the ones that work into your own toolkit and template them — a curated prompt library is genuine career capital.